

Laporan Praktikum Pemrograman Web

Jobsheet 7 dan 8



Oleh:

Muhammad Ramdhani Ariyadin (254107060018)

SIB 2-E

Link github :

<https://github.com/MuhammadRamdhaniAriyadin/Pemrograman-Web>

Jobshet 7

1. Tambah validasi ISBN di buku/proses_tambah.php — misalnya memastikan ISBN yang diisi (kalau tidak kosong) hanya berisi angka dan tanda hubung, memakai fungsi PHP `preg_match()`.
2. Tambah flash message di anggota/proses_tambah.php untuk kasus yang belum ditangani — bandingkan dengan versi buku/proses_tambah.php yang sudah divalidasi lebih lengkap (rentang tahun, stok non-negatif) — field apa lagi di form anggota yang mungkin perlu aturan validasi tambahan?
3. Buat halaman `debug_session.php` sementara (untuk latihan, hapus setelah selesai) yang menampilkan isi `$_SESSION` mentah lewat `<pre><?php print_r($_SESSION); ?></pre>` — cara yang berguna untuk "mengintip" langsung apa yang sebenarnya tersimpan di server saat belajar.
4. Tambah tombol "Reset Data" yang memanggil `session_destroy()` untuk mengosongkan seluruh `$_SESSION` secara manual, tanpa perlu menutup browser — cari tahu sendiri lewat dokumentasi PHP resmi bagaimana fungsi ini bekerja.

jawab

1. Saya telah menambahkan validasi server-side menggunakan fungsi `preg_match('/^[0-9-]+$/', $isbn)` untuk memastikan bahwa input ISBN (apabila diisi) hanya memuat karakter angka dan tanda hubung (-). Jika terdapat karakter lain, sistem akan membatalkan proses simpan dan menampilkan pesan error via flash message.

```
// validasi isbn
if ($isbn !== '' && !preg_match('/^[0-9-]+$/', $isbn)) {
    $errors[] = "ISBN hanya boleh berisi angka dan tanda hubung (-).";
}
```

Uji coba

Saya menginputkan isbn dengan huruf lalu aku simpan dan muncul kalimat eror



The screenshot shows a web form titled "Tambah Buku". At the top, there is a red error message box that says "ISBN hanya boleh berisi angka dan tanda hubung (-).". Below this, the form has four input fields labeled "Judul", "Pengarang", "Tahun Terbit", and "ISBN". The "ISBN" field is currently empty.

- Membandingkan dengan modul buku, saya menambahkan beberapa aturan validasi tambahan pada form anggota

nama: Memastikan panjang minimal 3 karakter dan hanya berisi huruf, spasi, titik, atau petik.

```
$errors = [];  
if ($nama === '') {  
    $errors[] = "Nama wajib diisi.";   
} elseif (strlen($nama) < 3) {  
    $errors[] = "Nama minimal 3 karakter.";   
} elseif (!preg_match('/^[a-zA-Z\s\.\']+$/', $nama)) {  
    $errors[] = "Nama hanya boleh berisi huruf, spasi, titik, dan petik.";   
}
```

Uji coba

The screenshot shows a web form titled "Tambah Anggota". Below the title, there is a pink error message box that reads: "Nama hanya boleh berisi huruf, spasi, titik, dan petik." Below this message, the label "Nama" is followed by an empty text input field.

no_anggota: Memastikan jika di isi hanya boleh berisi angka dan tanda hubung (-)

```
if ($noAnggota === '') {  
    $errors[] = "No. Anggota wajib diisi.";   
} elseif (!preg_match('/^[a-zA-Z0-9-]+$/', $noAnggota)) {  
    $errors[] = "No. Anggota hanya boleh berisi huruf, angka, dan tanda hubung (-).";   
}
```

Uji coba

The screenshot shows a web form titled "Tambah Anggota". Below the title, there is a pink error message box that reads: "No. Anggota hanya boleh berisi huruf, angka, dan tanda hubung (-)." Below this message, the label "Nama" is followed by an empty text input field.

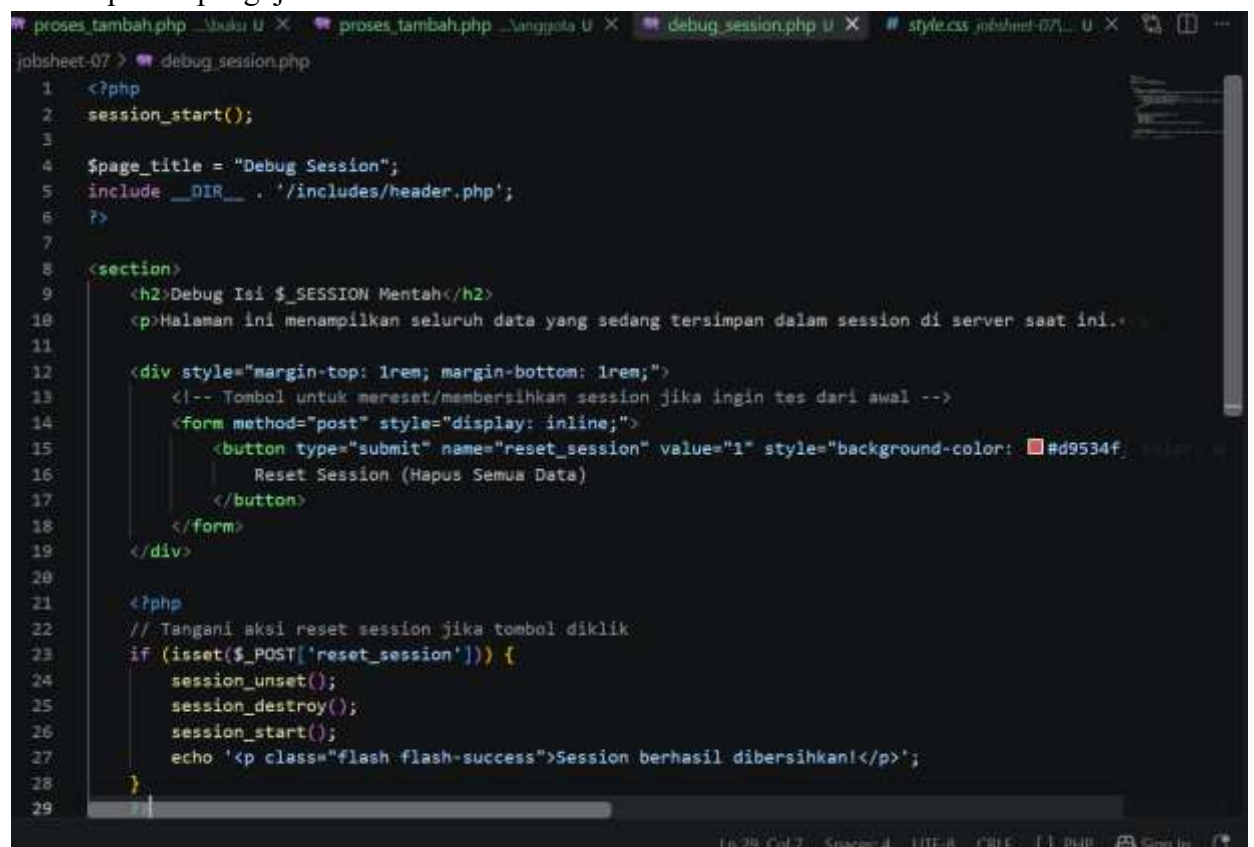
no_hp: Memastikan hanya memuat 10–15 digit angka (dengan opsional tanda + di awal).

```
if ($noHp !== '' && !preg_match('/^\+?[0-9]{10,15}$/', $noHp)) {  
    $errors[] = "No. HP harus berupa angka 10-15 digit (boleh diawali +).";  
}
```

Uji coba



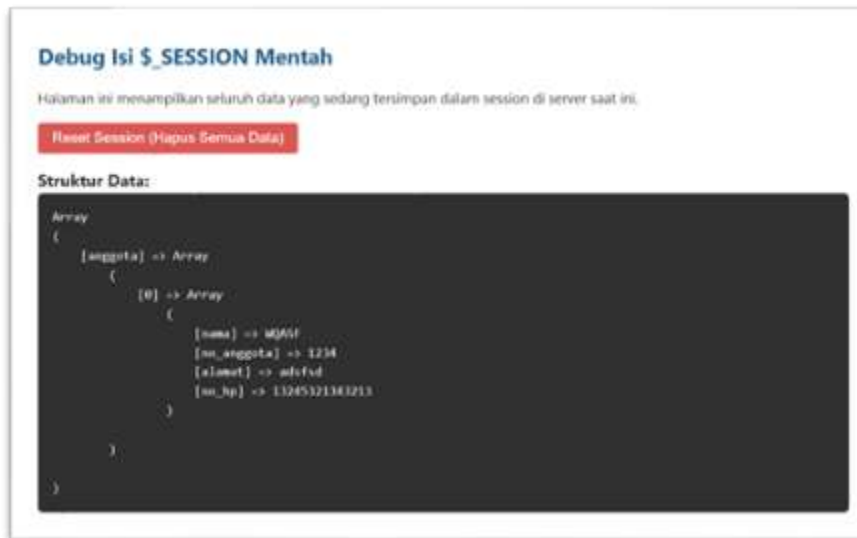
3. Pembuatan Halaman Debug Session (debug_session.php) Saya membuat file debug_session.php yang memanfaatkan tag `<pre><?php`
`print_r($_SESSION); ?></pre>` untuk menampilkan struktur data mentah yang tersimpan dalam `$_SESSION`. Halaman ini berfungsi sebagai alat pembantu (developer tool) untuk memantau status penyimpanan data secara real-time di server selama proses pengujian.



```
1 <?php  
2 session_start();  
3  
4 $page_title = "Debug Session";  
5 include __DIR__ . '/includes/header.php';  
6 ?>  
7  
8 <section>  
9     <h2>Debug Isi $_SESSION Mentah</h2>  
10    <p>Halaman ini menampilkan seluruh data yang sedang tersimpan dalam session di server saat ini.</p>  
11  
12    <div style="margin-top: 1rem; margin-bottom: 1rem;">  
13        <!-- Tombol untuk mereset/membersihkan session jika ingin tes dari awal -->  
14        <form method="post" style="display: inline;">  
15            <button type="submit" name="reset_session" value="1" style="background-color: #d9534f; color: white;">  
16                Reset Session (Hapus Semua Data)  
17            </button>  
18        </form>  
19    </div>  
20  
21    <?php  
22    // Tangani aksi reset session jika tombol diklik  
23    if (isset($_POST['reset_session'])) {  
24        session_unset();  
25        session_destroy();  
26        session_start();  
27        echo '<p class="flash flash-success">Session berhasil dibersihkan!</p>';  
28    }  
29
```

Uji Coba Tampilan Debug Session (debug_session.php)

Hasil: Halaman berhasil menampilkan isi data mentah \$_SESSION secara real-time, memperlihatkan array anggota yang berisi data input (nama, no. anggota, alamat, dan no. hp) yang tersimpan di server.



Uji Coba Tombol Reset Session

Hasil: Ketika tombol "Reset Session (Hapus Semua Data)" diklik, sistem berhasil menghapus seluruh session, menampilkan notifikasi flash message "Session berhasil dibersihkan!", dan mengosongkan struktur array \$_SESSION.



4. implementasi Tombol "Reset Data" (reset.php & header.php) Saya menambahkan fitur Reset Data pada navigasi utama. Fitur ini memanggil `session_unset()` untuk menghapus seluruh variabel session di memori dan `session_destroy()` untuk menghapus data session dari disk server sesuai rekomendasi dokumentasi resmi PHP. Dengan fitur ini, penyimpanan session dapat dikosongkan secara manual tanpa perlu menutup browser.

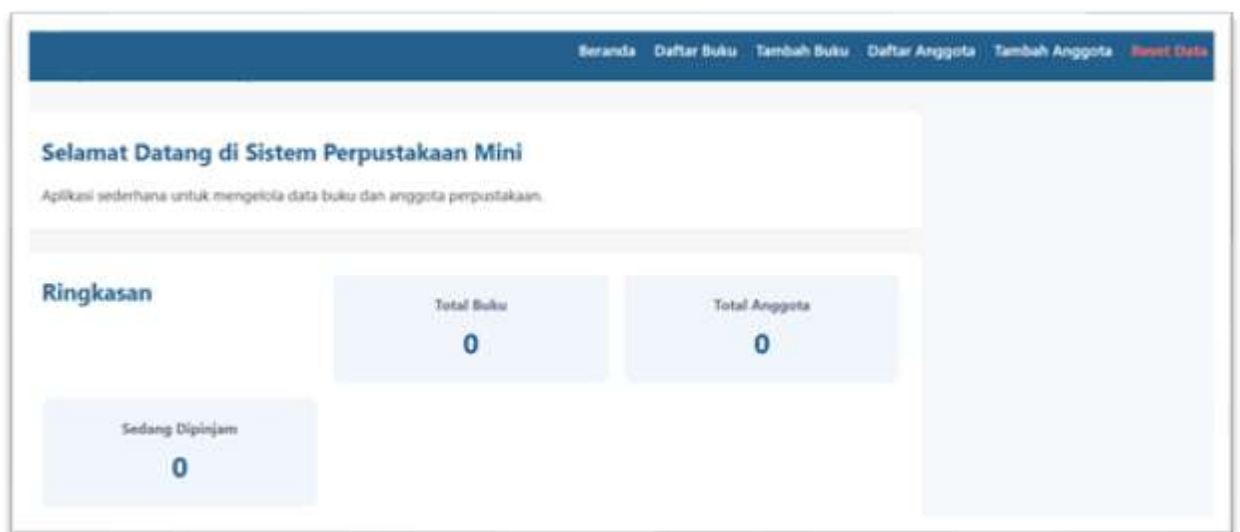
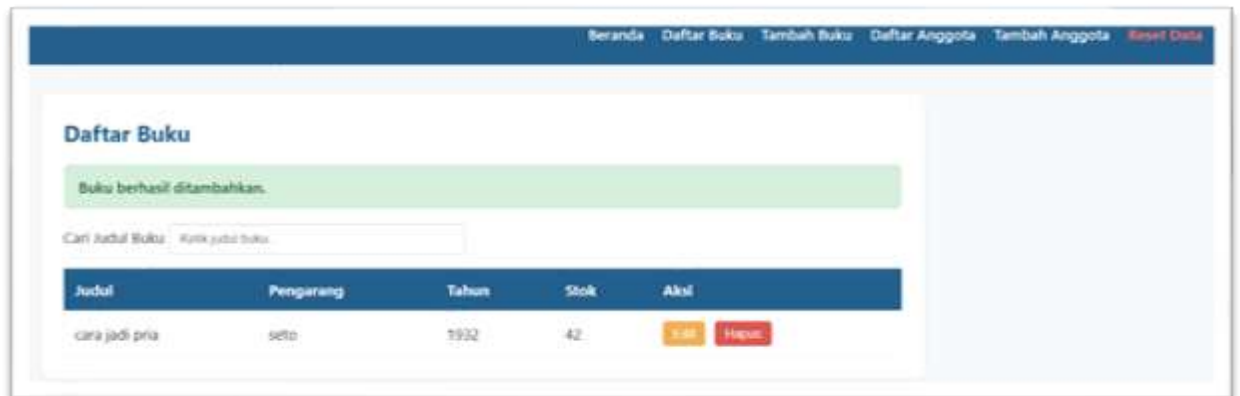
Membuat reset php

```
reset.php
1  <?php
2  session_start();
3
4  // 1. Kosongkan variabel
5  session_unset();
6
7  // 2. Hapus data
8  session_destroy();
9
10 // 3. Mulai session baru
11 session_start();
12 $_SESSION['flash'] = [
13     'type' => 'success',
14     'pesan' => 'Seluruh data session berhasil dikosongkan.'
15 ];
16
17 // 4. Redirect kembali ke beranda
18 header('Location: index.php');
19 exit;
```

Menambahkan tombol reset data di navigator bar

```
<!-- Tombol Reset Data dengan konfirmasi JavaScript -->
<li>
  <a href="<?php echo $base; ?>reset.php"
    onclick="return confirm('Apakah kamu yakin ingin menghapus SELURUH data session?');"
    style="color: #ff6b6b; font-weight: bold;">
    Reset Data
  </a>
```

Uji coba



Hasil Uji Coba Ketika tombol "Reset Data" pada navigasi diklik, fungsi `session_destroy()` berhasil menghapus seluruh data session. Terbukti dari ringkasan di Beranda (Total Buku dan Total Anggota) yang berubah dari berisi data menjadi 0.

Jobshet 8

1. Tangani error UNIQUE dengan rapi — bungkus \$stmt->execute(...) di anggota/proses_tambah.php dengan try/catch (PDOException \$e), lalu set \$_SESSION['flash'] berisi pesan seperti "No. Anggota sudah dipakai, gunakan nomor lain." alih-alih membiarkan error mentah ditampilkan ke pengguna.
2. Tambah kolom baru — misalnya tanggal_ditambahkan TIMESTAMP DEFAULT NOW() di tabel buku (cari tahu sendiri arti NOW() dan TIMESTAMP lewat dokumentasi PostgreSQL), lalu tampilkan kolom itu di buku/list.php.
3. Buat query pencarian di server — tambahkan WHERE judul ILIKE :keyword (ILIKE = pencocokan teks tanpa memandang huruf besar/kecil di PostgreSQL) ke query SELECT di buku/list.php, dihubungkan dengan kolom pencarian yang sudah ada di HTML — bandingkan dengan filter tabel sisi klien yang sudah kamu bangun di dokumentasi jobsheet-05 §6.
4. Migrasi data lama — coba tulis skrip PHP kecil terpisah yang membaca data/buku.json dari jobsheet-06 (dokumentasi jobsheet-06 §3) lalu memasukkan seluruh isinya ke tabel buku lewat INSERT — latihan bagus untuk memahami bagaimana data lama bisa "dipindahkan" ke database baru.

Jawab

1. Penanganan Error UNIQUE Constraint pada anggota/proses_tambah.php

Untuk mencegah aplikasi menampilkan pesan error mentah dari database saat terjadi duplikasi nomor anggota, eksekusi query \$stmt->execute(...) dibungkus menggunakan blok try...catch (PDOException \$e).

Jika terjadi error akibat pelanggaran integritas data (UNIQUE constraint dengan SQLSTATE 23000), sistem akan menangkap exception tersebut secara elegan, memasukkan pesan peringatan ke dalam \$_SESSION['flash'], dan mengarahkan (redirect) pengguna kembali ke halaman tambah.php

```
try {
    $stmt = $pdo->prepare(
        "INSERT INTO anggota (nama, no_anggota, alamat, no_hp)
        VALUES (:nama, :no_anggota, :alamat, :no_hp)
        RETURNING id"
    );
    $stmt->execute([
        'nama' => $nama,
        'no_anggota' => $noAnggota,
        'alamat' => $alamat,
        'no_hp' => $noHp,
    ]);
    $_SESSION['flash'] = ['type' => 'success', 'pesan' => 'Anggota berhasil ditambahkan.'];
    header('Location: list.php');
    exit;
} catch (PDOException $e) {
    // Checking SQLSTATE code 23000 (Integrity Constraint Violation / UNIQUE Error)
    if ($e->getCode() === '23000') {
        $_SESSION['flash'] = [
            'type' => 'error',
            'pesan' => 'No. Anggota sudah dipakai, gunakan nomor lain.'
        ];
    } else {
        $_SESSION['flash'] = [
            'type' => 'error',
            'pesan' => 'Gagal menambahkan anggota. Silakan coba lagi.'
        ];
    }
}
```

a. Pengujian Input Sukses (Data Unik)

Langkah:

Masukkan data anggota baru

Hasil yang Diharapkan:

Pengguna berhasil diarahkan ke halaman list.php.

Daftar Anggota

Cari Nama Anggota

No. Anggota	Nama	Alamat	No. HP	Aksi
1234	Ariyasthenics	malang	086776528912	<button>Edit</button> <button>Hapus</button>

b. Pengujian Duplikasi Data

Langkah:

Masukkan data anggota baru, namun gunakan No. Anggota yang sama persis dengan data sebelumnya (1234).

Tambah Anggota

Nama

ariya

No. Anggota

1234

Alamat

mojokerto

No. HP

123242132435

Simpan

Klik tombol Simpan / Submit.

Tambah Anggota

Gagal menambahkan anggota. Silakan coba lagi.

Nama

No. Anggota

Alamat

No. HP

Hasil yang Diharapkan:

Pengguna tidak melihat pesan error merah dari PHP/PostgreSQL (uncaught PDOException).

Pengguna secara rapi dikembalikan (redirect) ke halaman tambah.php.

Muncul pesan error flash sesuai ketentuan: "No. Anggota sudah dipakai, gunakan nomor lain."

2. Definisi Tipe Data & Fungsi PostgreSQL:

TIMESTAMP: Tipe data dalam PostgreSQL yang digunakan untuk menyimpan nilai tanggal dan waktu secara presisi (tahun, bulan, hari, jam, menit, detik).

NOW(): Fungsi bawaan PostgreSQL yang mengembalikan nilai tanggal dan waktu sistem saat ini (current timestamp).

DEFAULT NOW(): Klausula yang memastikan bahwa jika nilai kolom tidak diisi secara eksplisit saat proses INSERT, PostgreSQL secara otomatis mengisinya dengan waktu eksekusi query.

- Perubahan pada Database (DDL): Perintah SQL untuk menambahkan kolom tanggal_ditambahkan pada tabel buku:

```
alter table buku
add column tanggal_ditambahkan timestamp default now();
```

- Perubahan pada Kode (buku/list.php): Menambahkan header <th>Tanggal Ditambahkan</th> pada elemen <thead>, mengubah colspan menjadi 6, serta menampilkan nilainya pada elemen <tbody>:

```
<td>
<?php
echo !empty($buku['tanggal_ditambahkan'])
? date('d-m-Y H:i', strtotime($buku['tanggal_ditambahkan']))
: '-';
?>
</td>
```

- Pengujian

Masukkan data buku baru lalu simpan.

Buka halaman buku/list.php.

Hasil Diharapkan:

Data buku baru muncul di daftar.

Kolom Tanggal Ditambahkan terisi secara otomatis sesuai tanggal dan jam saat data tersebut dimasukkan tanpa perlu diinput manual dari form.

Daftar Buku					
Cari Judul Buku Ketik judul buku...					
Judul	Pengarang	Tahun	Stok	Tanggal Ditambahkan	Aksi
cara jadi pria	seto	1974	57	22-09-2026 20:56	<button>Edit</button> <button>Hapus</button>

- Implementasi Pencarian Sisi Server (Server-Side Search) Pencarian sisi server dilakukan dengan menambahkan kondisi WHERE judul ILIKE :keyword pada query SELECT di buku/list.php. Parameter pencarian ditangkap dari input HTML menggunakan metode GET (\$_GET['keyword']), kemudian nilai kata kunci dibungkus dengan karakter wildcard (%keyword%) agar dapat mencocokkan kata secara parsial.

Operator ILIKE merupakan operator khas PostgreSQL yang berfungsi melakukan pencocokan teks secara case-insensitive (mengabaikan perbedaan huruf besar dan huruf kecil).

```
// 1. Ambil keyword dari parameter GET URL
$keyword = trim($_GET['keyword'] ?? '');

// 2. Buat query pencarian dengan ILIKE
if ($keyword !== '') {
    $stmt = $pdo->prepare("SELECT * FROM buku WHERE judul ILIKE :keyword ORDER BY id DESC");
    $stmt->execute(['keyword' => "%$keyword%"]);
} else {
    $stmt = $pdo->prepare("SELECT * FROM buku ORDER BY id DESC");
    $stmt->execute();
}
```

Perbandingan Pencarian Sisi Server vs. Pencarian Sisi Klien (Jobsheet-05 §6)

Aspek Perbandingan	Server-Side Search (ILIKE / PostgreSQL)	Client-Side Search (JavaScript / Jobsheet-05 §6)
Lokasi Proses	Diproses langsung oleh database PostgreSQL di server.	Diproses oleh browser pengguna menggunakan JavaScript.
Performa & Skalabilitas	Sangat baik untuk data besar. Hanya data yang cocok yang dikirim dari database ke browser.	Hanya cocok untuk data kecil. Semua data harus di-render ke HTML terlebih dahulu baru difilter.
Sensitivitas Huruf	Otomatis <i>case-insensitive</i> berkat operator ILIKE.	Perlu penanganan manual pada kode JavaScript (misal: <code>.toLowerCase()</code>).

Aspek Perbandingan	Server-Side Search (ILIKE / PostgreSQL)	Client-Side Search (JavaScript / Jobsheet-05 §6)
Kebutuhan Reload	Membutuhkan <i>reload</i> halaman (atau request AJAX) untuk mengambil data baru.	Responsif dan <i>instant</i> tanpa perlu <i>reload</i> halaman.

Uji Coba Case Insensitivity (ILIKE):

- Masukkan kata pencarian dengan kombinasi huruf kapital tak beraturan (misal: pEmRoGrAmAn atau PEMROGRAMAN).
- **Hasil Diharapkan:** PostgreSQL tetap berhasil mencocokkan data karena operator `ILIKE` mengabaikan perbedaan huruf besar/kecil.

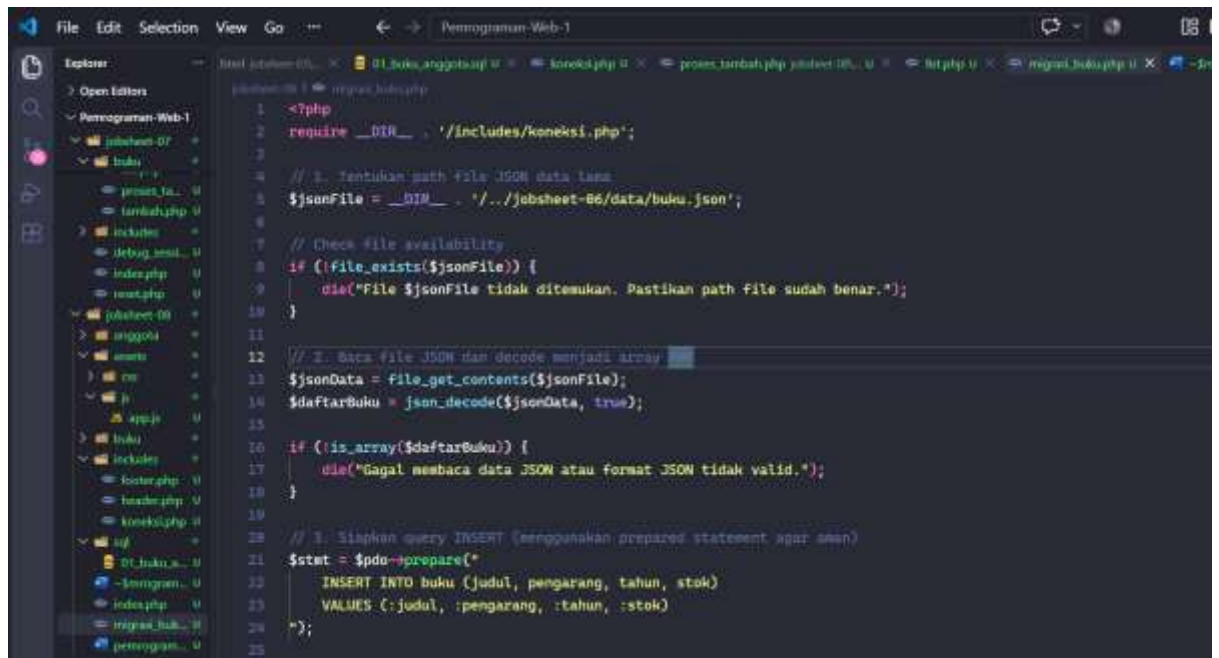


4. Proses migrasi data dari format berkas terstruktur (`data/buku.json`) ke dalam tabel database PostgreSQL dilakukan dengan cara membaca isi file JSON menggunakan `file_get_contents()`, mengonversinya menjadi *array associative* menggunakan `json_decode()`, lalu melakukan perulangan (`foreach`) untuk memasukkan tiap elemen ke dalam database melalui *prepared statement* `INSERT`.

Tujuan dan Manfaat:

Otomatisasi Pindahan Data: Mengidentifikasi struktur data lama dan memindahkannya ke skema database relasional tanpa perlu input manual satu per satu.

Keamanan & Integritas: Penggunaan prepared statement mencegah adanya kesalahan penulisan (syntax error) atau kebocoran query saat membaca string judul/pengarang dari berkas JSON.



```
1 <?php
2 require __DIR__ . '/includes/koneksi.php';
3
4 // 1. Tentukan path file JSON data lama
5 $jsonFile = __DIR__ . '/../jobsheet-06/data/buku.json';
6
7 // Check file availability
8 if (!file_exists($jsonFile)) {
9     die("File $jsonFile tidak ditemukan. Pastikan path file sudah benar.");
10 }
11
12 // 2. Baca file JSON dan decode menjadi array
13 $jsonData = file_get_contents($jsonFile);
14 $daftarBuku = json_decode($jsonData, true);
15
16 if (!is_array($daftarBuku)) {
17     die("Gagal membaca data JSON atau format JSON tidak valid.");
18 }
19
20 // 3. Siapkan query INSERT (menggunakan prepared statement agar aman)
21 $stmt = $pdo->prepare("
22     INSERT INTO buku (judul, pengarang, tahun, stok)
23     VALUES (:judul, :pengarang, :tahun, :stok)
24 ");
```

Langkah Uji Coba (Testing)

Persiapan Data:

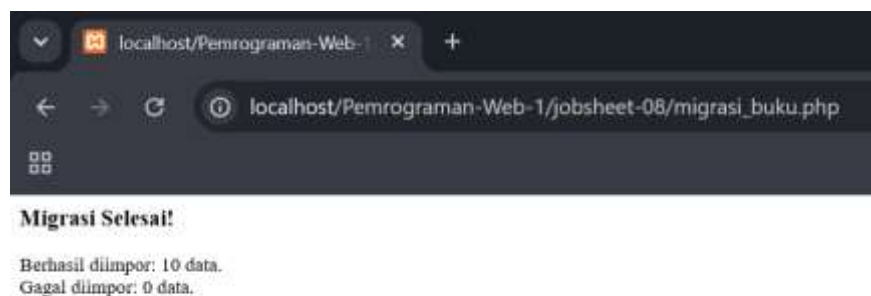
Pastikan file data/buku.json tersedia dan berisi array objek data buku.

Eksekusi Skrip Migrasi:

Akses skrip melalui browser (misal: http://localhost/proyek-kamu/migrasi_buku.php) atau jalankan via CLI terminal (`php migrasi_buku.php`).

Hasil Diharapkan:

Muncul ringkasan output status migrasi, contoh: "Migrasi Selesai! Berhasil diimpor: X data."



- Buka halaman `buku/list.php` atau cek via database client (pgAdmin/DBeaver), seluruh data dari file JSON lama telah masuk ke dalam tabel `buku` lengkap dengan penentuan nilai `tanggal_ditambahkan` otomatis dari PostgreSQL.

Daftar Buku

Cari Judul Buku

Judul	Pengarang	Tahun	Stok	Tanggal Ditambahkan	Aksi	
Gadis Kretek	Ratih Kumala	2012	4	22-09-2026 22:14	Edit	Hapus
Perahu Kertas	Dee Lestari	2009	0	22-09-2026 22:14	Edit	Hapus
Sang Pemimpi	Andrea Hirata	2006	6	22-09-2026 22:14	Edit	Hapus
Pulang	Tere Liye	2015	2	22-09-2026 22:14	Edit	Hapus
Cantik Itu Luka	Eka Kurniawan	2002	3	22-09-2026 22:14	Edit	Hapus
Ronggeng Dukuh Paruk	Ahmad Tohari	1982	1	22-09-2026 22:14	Edit	Hapus
Filosofi Teras	Henry Manampiring	2018	5	22-09-2026 22:14	Edit	Hapus
Negeri 5 Menara	Ahmad Fuadi	2009	0	22-09-2026 22:14	Edit	Hapus
Bumi Manusia	Pramoedya Ananta Toer	1980	2	22-09-2026 22:14	Edit	Hapus
Laskar Pelangi	Andrea Hirata	2005	4	22-09-2026 22:14	Edit	Hapus